

Machine Learning

Volker Roth

Department of Mathematics & Computer Science
University of Basel

Chapter 8: Gaussian Processes

- The use of the Gaussian distribution in ML
 - ▶ Properties of the **multivariate Gaussian distribution**
 - ▶ Random variables $\rightarrow$ random vectors $\rightarrow$ **stochastic processes**
 - ▶ **Gaussian processes for regression**
 - ▶ **Model Selection**
 - ▶ **Gaussian processes for classification**
- Relation to **kernel models** (e.g. SVMs)
- Relation to **neural networks**.

Kernel Ridge Regression

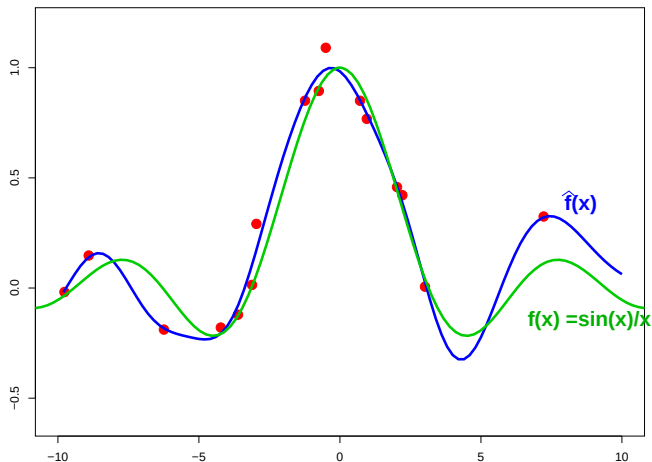
- **Kernelized ridge regression:** $\hat{\mathbf{w}} = (X^t X + \lambda I)^{-1} X^t \mathbf{y}$.
- Matrix inversion lemma: $(I + UV)^{-1} U = U(I + VU)^{-1}$
- Define new variables α_i :

$$\begin{aligned}\hat{\mathbf{w}} &= (X^t X + \lambda I)^{-1} X^t \mathbf{y} \\ &= X^t \underbrace{(X X^t + \lambda I)^{-1}}_{\hat{\alpha}} \mathbf{y} = \sum_{i=1}^n \hat{\alpha}_i \mathbf{x}_i.\end{aligned}$$

- **Predictions for new $\mathbf{x}_*$:**

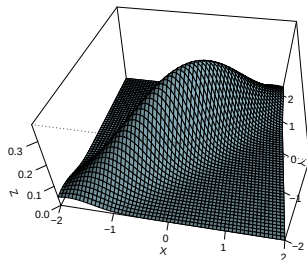
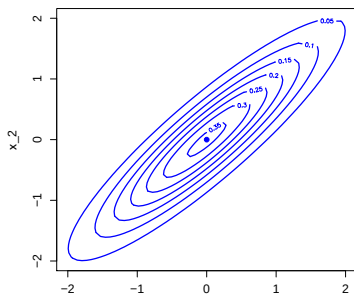
$$\hat{f}(\mathbf{x}_*) = \hat{\mathbf{w}}^t \mathbf{x}_* = \sum_{i=1}^n \hat{\alpha}_i \mathbf{x}_i^t \mathbf{x}_* = \sum_{i=1}^n \hat{\alpha}_i k(\mathbf{x}_i, \mathbf{x}_*).$$

Kernel Ridge Regression



Kernel function: $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\frac{1}{2l^2} \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

How can we make use of the Gaussian distribution?



- Is it possible to fit a **nonlinear regression line** with the “boring” Gaussian distribution?
- **Yes**, but we need to introduce the concept of **Gaussian Processes**!

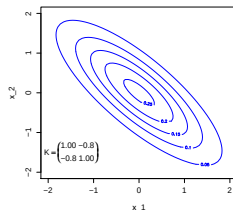
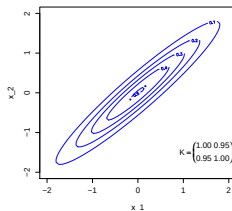
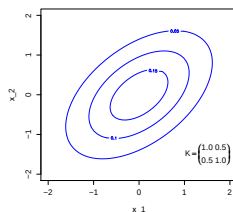
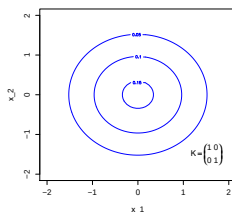
The 2D Gaussian distribution

$$\text{2D Gaussian: } P(\mathbf{y}; \boldsymbol{\mu} = \mathbf{0}, \Sigma = K) = \frac{1}{\sqrt{2\pi|K|}} \exp(-\frac{1}{2}\mathbf{y}^t K^{-1} \mathbf{y})$$

Covariance

(also written “co-variance”) is a measure of how much **two random variables vary together**:

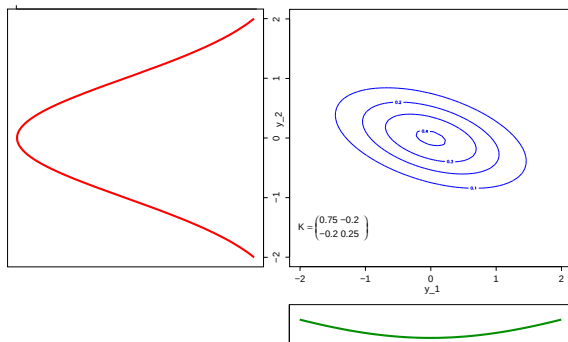
- **+1**: perfect linear coherence,
- **-1**: perfect negative linear coherence,
- **0**: no linear coherence.



Properties of the Multivariate Gaussian distribution

$\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}, K)$. Let $\mathbf{y} = \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{pmatrix}$ and $K = \begin{pmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{pmatrix}$.

Then $\mathbf{y}_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, K_{11})$ and $\mathbf{y}_2 \sim \mathcal{N}(\boldsymbol{\mu}_2, K_{22})$.

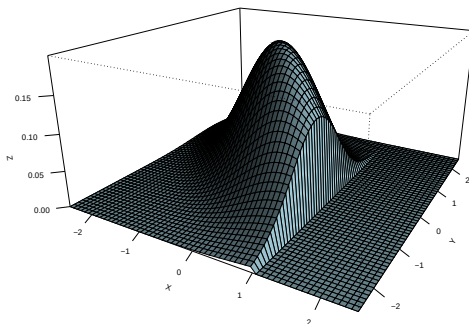


Marginals of Gaussians are again Gaussian!

Properties of the Multivariate Gaussian distribution (2)

$\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}, K)$. Let $\mathbf{y} = \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{pmatrix}$ and $K = \begin{pmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{pmatrix}$.

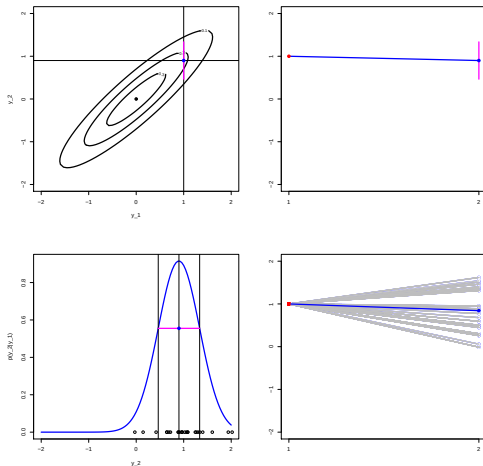
Then $\mathbf{y}_2 | \mathbf{y}_1 \sim \mathcal{N}(\boldsymbol{\mu}_2 + K_{21}K_{11}^{-1}(\mathbf{y}_1 - \boldsymbol{\mu}_1), K_{22} - K_{21}K_{11}^{-1}K_{12})$.



Conditionals of Gaussians are again Gaussian!

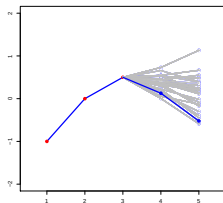
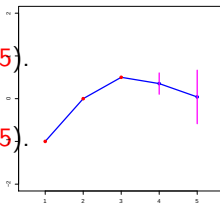
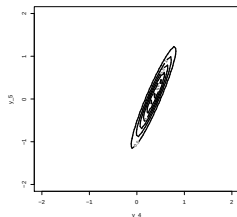
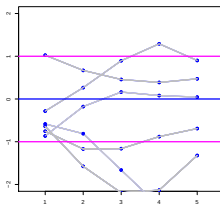
2D Gaussians: a new visualization

- **top left:** mean and $\pm$ std.dev. of $p(y_2|y_1 = 1)$.
- **bottom left:** $p(y_2|y_1 = 1)$ and samples drawn from it.
- **top right:** x-axis: indices (1, 2) of dimensions, y-axis: density in each component. Shown are $y_1 = 1$ and the conditional mean $\bar{p}(y_2|y_1 = 1)$ and std.dev.
- **bottom right:** samples drawn from above model.



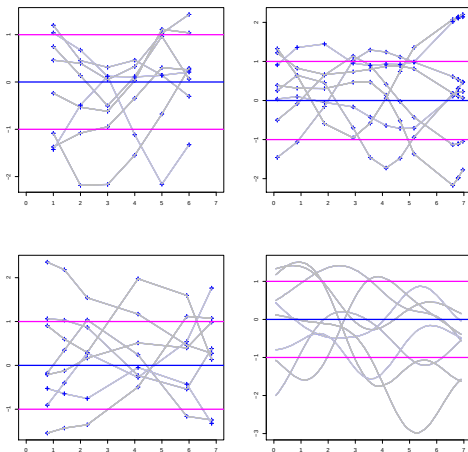
Visualizing high-dimensional Gaussians

- **top left:** 6 samples drawn from 5-dimensional Gaussian with zero mean (indicated by blue line). $\sigma = 1$ (magenta line).
- **bottom left:** Conditional mean and std.dev of $p(y_4, y_5 | y_1 = -1, y_2 = 0, y_3 = 0.5)$.
- **top right:** contour lines of $p(y_4, y_5 | y_1 = -1, y_2 = 0, y_3 = 0.5)$.
- **bottom right:** samples drawn from above model.

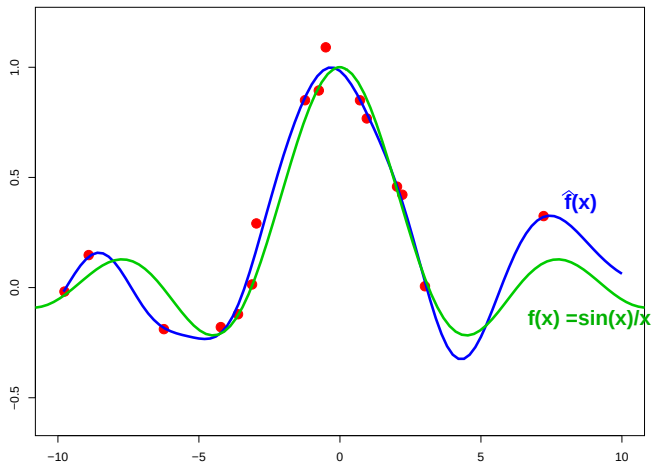


From covariance matrices to Gaussian processes

- **top left:** 8 samples, 6 dim.
x-axis: dimension-indices.
- **bottom left:** 8 samples,
viewed as values $y = f(\mathbf{x})$.
Construction: choose 6
input points $\mathbf{x}_i$ at random
 $\rightsquigarrow$ build covariance matrix K
with **covariance function**
 $k(\mathbf{x}, \mathbf{x}') = \exp(-\frac{1}{2l^2}\|\mathbf{x} - \mathbf{x}'\|^2)$
 $\rightsquigarrow$ draw $\mathbf{f} \sim \mathcal{N}(\mathbf{0}, K)$
 $\rightsquigarrow$ plot as function of inputs.
- **top right:** same for 12 inputs
- **bottom right:** 100 inputs



This looks similar to Kernel Regression...



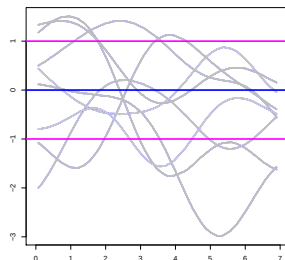
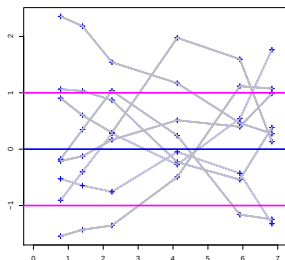
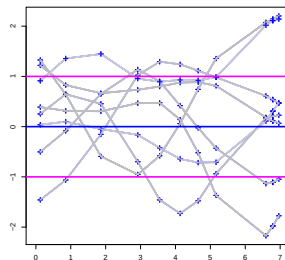
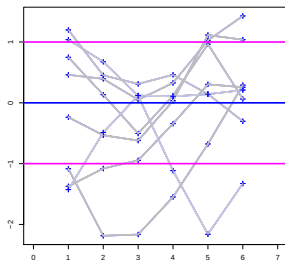
Gaussian Processes

- Gaussian **Random Variable** (RV): $f \sim \mathcal{N}(\mu, \sigma^2)$.
- Gaussian **Random Vector**: Collection of n RVs, characterized by mean vector and covariance matrix: $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$
- Gaussian **Process**: infinite Gaussian random vector, every finite subset of which is jointly Gaussian distributed
Continuous index, e.g. time $t \rightsquigarrow$ **function** $f(t)$.
Fully specified by **mean function** $m(t) = \mathbb{E}[f(t)]$
and **covariance function** $k(t, t') = \mathbb{E}[(f(t) - m(t))(f(t') - m(t'))]$.
- In ML, we will focus on more general index sets $\mathbf{x} \in \mathbb{R}^d$ with **mean function** $m(\mathbf{x})$ and **covariance function** $k(\mathbf{x}, \mathbf{x}')$:
$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

Visualizing Gaussian Processes: Sampling

- **Problem:** working with infinite vectors and covariance matrices is not very intuitive...
- **Solution:** evaluate the GP at set of n discrete times (or input vectors $\mathbf{x} \in \mathbb{R}^d$):
 - ▶ Choose n **input points** $\mathbf{x}_i$ at random $\rightsquigarrow$ matrix X
 - ▶ build **covariance matrix** $K(X, X)$ with **covariance function** $k(\mathbf{x}_i, \mathbf{x}_j)$
 - ▶ **sample** realizations of the Gaussian random vector $\mathbf{f} \sim \mathcal{N}(\mathbf{0}, K(X, X))$
 - ▶ plot $\mathbf{f}$ as **function of inputs**.

This is exactly what we have done here...



From the Prior to the Posterior

GP defines distribution over functions $\rightsquigarrow \mathbf{f}$ evaluated at training points X and $\mathbf{f}_*$ evaluated at test points X_* are jointly Gaussian:

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K(X, X) & K(X, X_*) \\ K(X_*, X) & K(X_*, X_*) \end{bmatrix}\right)$$

Posterior $p(\mathbf{f}_* | X_*, X, \mathbf{f}(X))$: conditional of a Gaussian distribution.

Let $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, K)$. Let $\mathbf{x} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{pmatrix}$ and $K = \begin{pmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{pmatrix}$.
Then $\mathbf{x}_2 | \mathbf{x}_1 \sim \mathcal{N}(\boldsymbol{\mu}_2 + K_{21}K_{11}^{-1}(\mathbf{f}_1 - \boldsymbol{\mu}_1), K_{22} - K_{21}K_{11}^{-1}K_{12})$.

$$\mathbf{f}_* | X_*, X, \mathbf{f} \sim \mathcal{N}\left(\begin{aligned} &K(X_*, X)(K(X, X))^{-1}\mathbf{f}, \\ &K(X_*, X_*) - K(X_*, X)(K(X, X))^{-1}K(X, X_*) \end{aligned}\right)$$

For only one test case:

$$f_* | \mathbf{x}_*, X, \mathbf{f} \sim \mathcal{N}(\mathbf{k}_*^t K^{-1} \mathbf{f}, k_{**} - \mathbf{k}_*^t K^{-1} \mathbf{k}_*)$$

A simple extension: noisy observations

- Assume we have access only to noisy versions of function values:
 $y = f(\mathbf{x}) + \eta$, $\eta \sim \mathcal{N}(0, \sigma^2)$ (cf. initial example of **ridge regression**).
- Noise η does not depend on data!
- Covariance of noisy observations $\mathbf{y}$ is sum of covariance of f and variance of noise: $\text{cov}(\mathbf{y}) = K(X, X) + \sigma^2 I$.

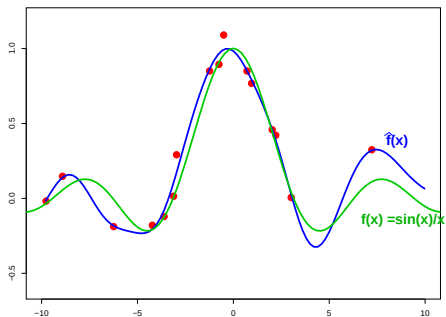
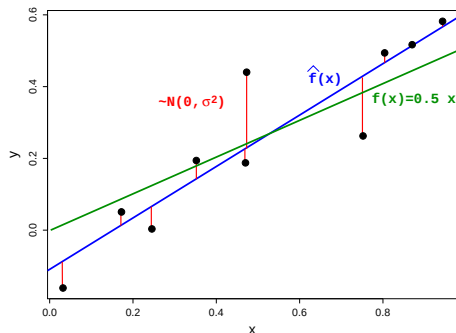
$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K(X, X) + \sigma^2 I & K(X, X_*) \\ K(X_*, X) & K(X_*, X_*) \end{bmatrix}\right)$$

$$\mathbf{f}_* | X_*, X, \mathbf{y} \sim \mathcal{N}\left(\begin{array}{cc} K(X_*, X)(K(X, X) + \sigma^2 I)^{-1} \mathbf{y}, \\ K(X_*, X_*) - K(X_*, X)(K(X, X) + \sigma^2 I)^{-1} K(X, X_*) \end{array}\right)$$

$$f_* | \mathbf{x}_*, X, \mathbf{y} \sim \mathcal{N}(\mathbf{k}_*^t (K + \sigma^2 I)^{-1} \mathbf{y}, k_{**} - \mathbf{k}_*^t (K + \sigma^2 I)^{-1} \mathbf{k}_*)$$

$\Rightarrow$ Posterior mean is solution of **kernel ridge regression**!

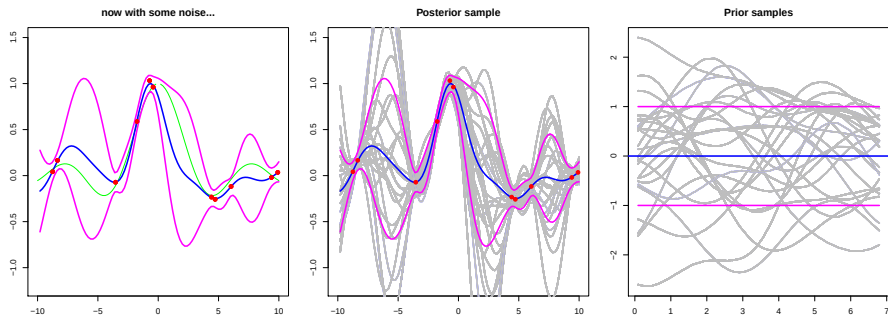
Noisy observations: examples



Noisy observations: $y = f(x) + \eta$, $\eta \sim \mathcal{N}(0, \sigma^2)$

Mean predictions: $\hat{f}_* = K_*(K + \sigma^2 I)^{-1} \mathbf{y}$.

Gaussian processes for regression



- **Left:** 11 training points generated as $y = \sin(x)/x + \nu$, $\nu \sim \mathcal{N}(0, 0.01)$
Covariance $k(\mathbf{x}_p, \mathbf{x}_q) = \exp(-\frac{1}{2l^2}\|\mathbf{x}_p - \mathbf{x}_q\|^2) + \sigma^2\delta_{p,q}$.
100 test points uniformly chosen from $[-10, 10] \rightsquigarrow$ matrix X_* .
Mean prediction $E[\mathbf{f}_*|X_*, X, \mathbf{y}]$ and $\pm\text{std.dev.}$
- **Middle:** samples drawn from posterior $\mathbf{f}_*|X_*, X, \mathbf{y}$.
- **Right:** samples drawn from prior $\mathbf{f} \sim \mathcal{N}(\mathbf{0}, K(X, X))$.

Covariance Functions

- A GP specifies a distribution over functions $f(\mathbf{x})$, characterized by mean function $m(\mathbf{x})$ and covariance function $k(\mathbf{x}_i, \mathbf{x}_j)$.

- Finite subset evaluated at n inputs $\rightsquigarrow$ Gaussian distribution:

$$\mathbf{f}(X) = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_n))^t \sim \mathcal{N}(\boldsymbol{\mu}, K),$$

where K is the covariance matrix with entries $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

- Covariance matrices are **symmetric positive semi-definite**:

$$K_{ij} = K_{ji} \text{ and } \mathbf{x}^t K \mathbf{x} \geq 0, \forall \mathbf{x}.$$

- We already know that **Mercer kernels** have this property $\rightsquigarrow$ all Mercer kernels define proper covariance functions in GPs.
- Kernels frequently have **additional parameters**.
- The **noise variance** in the observation model $y = f(\mathbf{x}) + \eta, \eta \sim \mathcal{N}(0, \sigma^2)$ is another parameter.
- How should we choose these parameters? $\rightsquigarrow$ **model selection**.

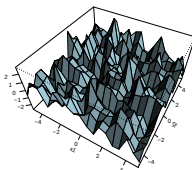
Model Selection

- **top left:** sample function from prior $\mathbf{f} \sim \mathcal{N}(\mathbf{0}, K(\mathbf{X}, \mathbf{X}))$ with **covariance function**

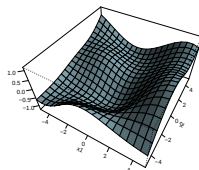
$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{1}{2l^2} \|\mathbf{x} - \mathbf{x}'\|^2\right).$$

Length scale $l = 10^{-0.5}$ small

$\rightsquigarrow$ highly varying function.

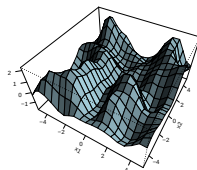


length scale: $10^{-0.5}$, sample no. 1



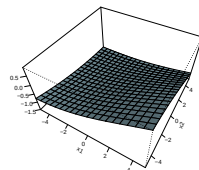
length scale: 10^0 , sample no. 1

- **bottom left:** same for $l = 10^0$
 $\rightsquigarrow$ smoother function



length scale: 10^0 , sample no. 1

- **top right:** same for $l = 10^{0.5}$
 $\rightsquigarrow$ even smoother...



length scale: $10^{0.5}$, sample no. 1

- **bottom right:** almost linear function for $l = 10^1$.

Model Selection (2)

- How to select the parameters?
- One possibility: maximize **marginal likelihood**:

$$p(\mathbf{y}|X) = \int p(\mathbf{y}|\mathbf{f}, X)p(\mathbf{f}|X) d\mathbf{f}.$$

- We do not need to integrate: we know that

$$\mathbf{f}|X \sim \mathcal{N}(\mathbf{0}, K) \quad \text{and} \quad \mathbf{y} = \mathbf{f} + \eta, \quad \eta \sim \mathcal{N}(0, \sigma^2).$$

Since η does not depend on X , the variances simply add:

$$\mathbf{y}|X \sim \mathcal{N}(\mathbf{0}, K + \sigma^2 I).$$

- Possible strategy:
Select parameters on a grid and choose maximum.
- Or: Compute derivatives of marginal likelihood and use gradient descent.

Model Selection (3)

- **Example problem:** $y = \sin(x)/x + \eta$, $\eta \sim \mathcal{N}(0, 0.01)$.

- **Log marg. likeli.** $= \log \mathcal{N}(\mathbf{0}, K + \sigma^2 I) =$

$$\underbrace{-\frac{1}{2} \mathbf{y}^t (K + \sigma^2 I)^{-1} \mathbf{y}}_{\text{data fit}} - \underbrace{\frac{1}{2} \log |K + \sigma^2 I|}_{\text{complexity penalty}} - \underbrace{\frac{n}{2} \log(2\pi)}_{\text{norm. constant}} .$$

- 2d-Example with Gaussian RBF:

$$(K + \sigma^2 I) = \begin{pmatrix} 1 + \sigma^2 & a \\ a & 1 + \sigma^2 \end{pmatrix} \Rightarrow |K + \sigma^2 I| = (1 + \sigma^2)^2 - a^2 > 0$$

Note that $a \rightarrow 0$ if length scale $l \rightarrow 0$

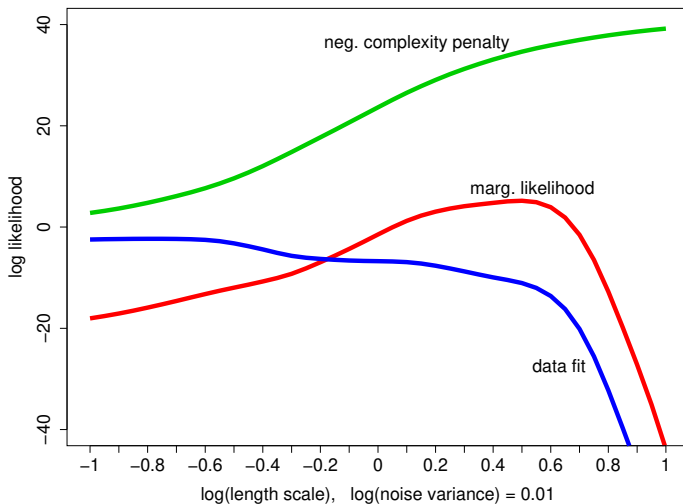
$\leadsto$ complexity penalty has high values for small length scales.

Matrix inverse includes a dominating factor $|K + \sigma^2 I|^{-1}$

$\leadsto$ data fit term also high for small l .

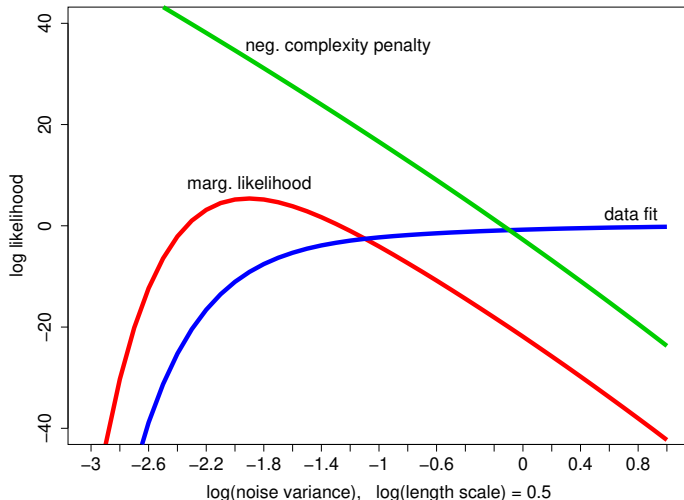
Model Selection (4)

Fixing $\sigma^2 = 0.01$ and **varying length scale** l :



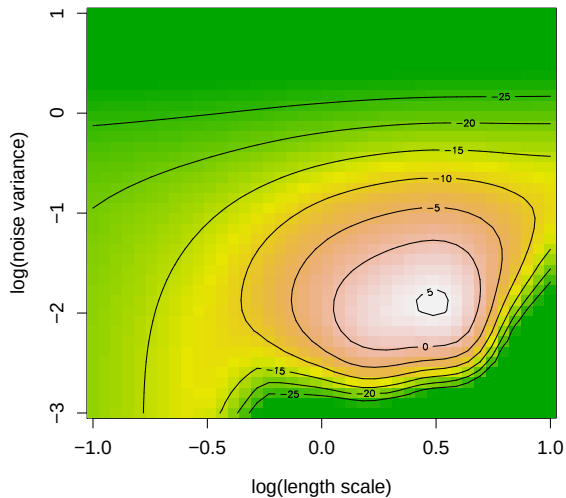
Model Selection (5)

Fixing length scale $\log(l) = 0.5$ and **varying the noise level σ^2** :

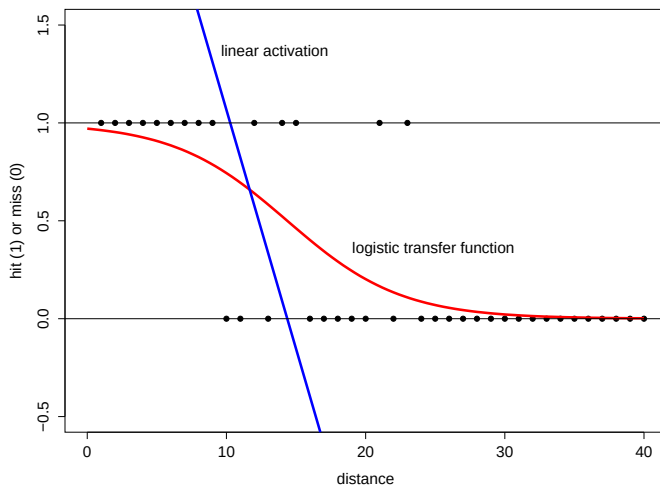


Model Selection (6)

Varying both σ^2 and l :



Classification: Basket Ball Example



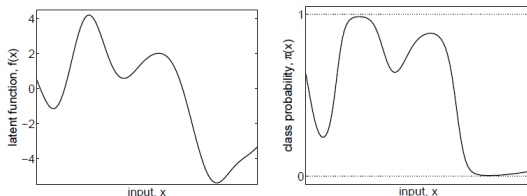
Adapted from Fig. 7.5.1 in B. Flury: A first course in multivariate statistics. Springer 1997.

Classical Logistic Regression

- Targets $y \in \{0, 1\}$
 $\rightsquigarrow$ Bernoulli RV with “success probability” $\pi(\mathbf{x}) = P(1|\mathbf{x})$.
- Likelihood: $P(\mathbf{y}|X, f) = \prod_{i=1}^n (\pi_f(\mathbf{x}_i))^{y_i} (1 - \pi_f(\mathbf{x}_i))^{1-y_i}$
- **Linear logistic regression:** unbounded $f(\mathbf{x}) = \mathbf{w}^t \mathbf{x}$ (“activation”)
Bounded estimates: pass $f(\mathbf{x})$ through **logistic transfer function**
 $\sigma(f(\mathbf{x})) = \frac{e^{f(\mathbf{x})}}{1+e^{f(\mathbf{x})}} = \frac{1}{1+e^{-f(\mathbf{x})}}$ and set $\pi_f(\mathbf{x}) = \sigma(f(\mathbf{x}))$.
- Use gradient-based methods for finding $\hat{\mathbf{w}}$ that maximizes the log posterior.
- Kernel trick: expand $\mathbf{w} = X^t \alpha$, substitute dot products by kernel function $k(\mathbf{x}, \mathbf{x}') \rightsquigarrow$ **kernel logistic regression**.

GP Classification

- Place GP prior over “latent” function $\mathbf{f}(\mathbf{x}) \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}'))$.
- “Squash” it through logistic function $\rightsquigarrow$ prior on $\pi(\mathbf{x}) = \sigma(f(\mathbf{x}))$.



(Rasmussen & Williams, 2006)

- **Problem:** Bernoulli likelihood $\rightsquigarrow$ predictive distribution $p(y_* = 1 | X, \mathbf{y}, \mathbf{x}_*)$ cannot be calculated analytically.
- Possible **solution:** use **Laplace approximation** of posterior $p(\mathbf{f} | X, \mathbf{y})$.

GP Classification using Laplace's approximation

- Prior $\mathbf{f}|X \sim \mathcal{N}(\mathbf{0}, K)$. Bernoulli likelihood:

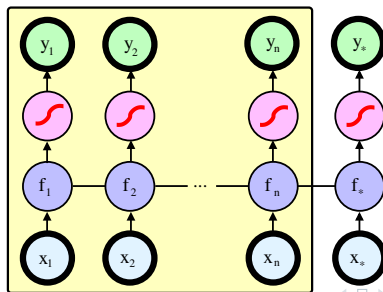
$$p(\mathbf{y}|X, \mathbf{f}) = \prod_{i=1}^n (\sigma(f(\mathbf{x}_i)))^{y_i} (1 - \sigma(f(\mathbf{x}_i)))^{1-y_i}.$$

- Gaussian approximation of posterior:

$$p(\mathbf{f}|X, \mathbf{y}) \approx \mathcal{N}(\hat{\mathbf{f}}, H^{-1}).$$

- Predictions: compute

$$p(y_* = 1|\mathbf{y}, \mathbf{x}_*, X) = \int \sigma(f_*) \underbrace{p(f_* | \mathbf{y}, \mathbf{x}_*, X)}_{\text{latent function at } \mathbf{x}_*} df_* = \mathbb{E}_{p(f_*|\mathbf{y}, \mathbf{x}_*, X)}(\sigma)$$



GP Classification using Laplace's approximation

- First **predict latent function** at test case $\mathbf{x}_*$:

$$\begin{aligned} p(f_* | \mathbf{y}, \mathbf{x}_*, X) &= \int \underbrace{p(f_* | \mathbf{f}, \mathbf{x}_*, X)}_{\text{Gaussian}} \underbrace{p(\mathbf{f} | X, \mathbf{y}) d\mathbf{f}}_{\text{approx. Gaussian } \mathcal{N}(\hat{\mathbf{f}}, H^{-1})} \\ &\approx \mathcal{N}(\mu_*, \sigma_*), \text{ with} \\ \mu_* &= \mathbf{k}_*^t K^{-1} \hat{\mathbf{f}}, \\ \sigma_* &= k_{**} - \mathbf{k}_*^t \tilde{K}^{-1} \mathbf{k}_* \end{aligned}$$

- Then use **Monte Carlo approximation**

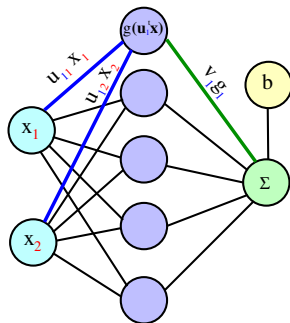
$$p(y_* | \mathbf{y}, \mathbf{x}_*, X) = \mathbb{E}_{p(f_* | \mathbf{y}, \mathbf{x}_*, X)}(\sigma) \approx \frac{1}{S} \sum_{s=1}^S \sigma(f_*^s(\mathbf{x}_*)),$$

where f_*^s are samples from the (approximated) distribution over latent function values.

GPs and Neural networks

Consider a neural network with one hidden layer of n_H units:

$$f(\mathbf{x}) = b + \sum_{j=1}^{n_H} v_j g(\mathbf{x}; \mathbf{u}_j).$$



- **Bayesian treatment:** All weights are considered as **RVs** for which we define **prior distributions**: zero-mean Gaussian priors for b and v , and for components of the weight vector $\mathbf{u}_j$ at the j -th hidden unit.
- What is the asymptotic distribution of $f(\mathbf{x})$, as $n_H \rightarrow \infty$?

GPs and Neural networks

NN-GP kernel (Neal 1996, Williams 1998)

A MLP with one hidden layer of infinite width, which has Gaussian priors on all parameters, converges to a Gaussian process with a well-defined covariance function (extensions to deep MLPs, CNNs etc. exist).

- Network with n_H hidden units implements function $f(\mathbf{x}) = b + \sum_{j=1}^{n_H} v_j g(\mathbf{x}; \mathbf{u}_j)$, $g(\mathbf{x}; \mathbf{u}_j) = \varphi(u_{j0} + \mathbf{x}^t \mathbf{u}_j)$.
- Priors: $b \sim N(0, \sigma_b^2)$, $v_j \sim N(0, \sigma_v^2)$, $\mathbf{u}_j \sim N(\mathbf{0}, \Sigma_u) \forall j = 1 \dots, n_H$.
- Let $\theta = \{b, v_j, \mathbf{u}_j\}$ be all the parameters.

Mean of network output:

$$\mathbb{E}_{\theta}[f(\mathbf{x})] = \overbrace{\mathbb{E}_{\theta}[b]}^{=0} + \sum_{j=1}^{n_H} \mathbb{E}_{\theta}[v_j g(\mathbf{x}; \mathbf{u}_j)]$$
$$\underset{(\text{v indep. of } u)}{=} \sum_{j=1}^{n_H} \underbrace{\mathbb{E}_{\theta}[v_j]}_{=0} \mathbb{E}_u[g(\mathbf{x}; \mathbf{u}_j)] = 0.$$

GPs and Neural networks: Covariance

Covariance when the function is applied to two inputs:

$$\begin{aligned}\mathbb{E}_{\theta}[f(\mathbf{x})f(\mathbf{x}')] &= \mathbb{E}_{\theta}[(b + \sum_{j=1}^{n_H} v_j g_j(\mathbf{x}))(b + \sum_{j=1}^{n_H} v_j g_j(\mathbf{x}'))] \\ &= \sigma_b^2 + \sum_{j=1}^{n_H} \mathbb{E}_{\theta}[v_j^2] \mathbb{E}_{\mathbf{u}}[g(\mathbf{x}; \mathbf{u}_j)g(\mathbf{x}'; \mathbf{u}_j)] \\ &= \sigma_b^2 + \sigma_v^2 n_H \mathbb{E}_{\mathbf{u}}[g(\mathbf{x}; \mathbf{u})g(\mathbf{x}'; \mathbf{u})],\end{aligned}$$

because all of the hidden units are identically distributed.

- Let $n_H \rightarrow \infty$. Scale magnitude of output by defining $\sigma_v^2 = \frac{\omega}{n_H}$.
- Input to the output neuron is an **infinite sum over i.i.d. RVs**.
 $\rightsquigarrow$ **central limit theorem** $\rightsquigarrow$ for a single input $\mathbf{x}$: Gaussian distribution $f(\mathbf{x}) \sim N(0, \sigma_b^2 + \omega \mathbb{E}_{\mathbf{u}}[g(\mathbf{x})^2])$
- Collection of n inputs $\mathbf{x}_i$ $\rightsquigarrow$ joint distribution: multivariate zero-mean Gaussian with covariance $\sigma_b^2 + \omega \mathbb{E}_{\mathbf{u}}[g(\mathbf{x})g(\mathbf{x}')] := k_{\text{NN-GP}}(\mathbf{x}, \mathbf{x}')$.
- For some activations φ , NN-GP kernel can be computed analytically.

Summary

- **GPs: fully probabilistic models**
 $\leadsto$ posterior $p(\mathbf{f}_* | X, \mathbf{y}, \mathbf{x}_*)$.
- Uniquely defined by specifying **covariance function**.
- **Mathematically simple:**
we only need to calculate **conditionals of Gaussians!**
- **Connections:**
regression: $\text{MAP}(\text{GP}_r) = \text{kernel ridge reg.}$
 $\text{GP}_c \approx$ probabilistic version of SVM.

Networks of infinite width can be interpreted as

Gaussian Processes with the NN-GP kernel.

It is also possible to derive kernels from networks **after training:**

Neural Tangent Kernel.